

Mixdog 사용보고서

같은 모델과 예산으로 더 많은 작업을 하기 위한 시작·사용 안내

대상: Mixdog 사용자 · 근거: README.md, package.json · 제품 버전: 0.9.165

요지

Mixdog는 가장 효율적인 AI 코딩 하네스를 가장 쓰기 쉽게 만드는 제품이다. 캐시를 의식한 컨텍스트, 초점을 맞춘 도구, compaction으로 오버헤드를 줄여 같은 모델과 예산이 작업 자체에 더 쓰이게 한다. Desktop-CLI에서 코드를 고치고, 브라우저·Windows 앱·문서·이미지·영상까지 같은 세션으로 이어갈 수 있다.

1. 시작 방법

Windows는 Desktop 설치 파일로 시작할 수 있다. 현재 설치 파일은 서명되지 않아¹ Windows SmartScreen 경고가 나올 수 있다. CLI는 Node.js 22.19 이상(22.x 또는 24+)에서 `npm install -g mixdog` 후 mixdog를 실행한다. 첫 실행이 제공자 인증, 모델 선택, 워크플로 설정을 안내한다.

CLI 네이티브 자산은 Windows x64, macOS x64/ARM64, Linux x64/ARM64를 지원한다. Windows ARM64 Node.js는 지원하지 않으며, ARM Windows에서는 x64 Node.js 에뮬레이션을 쓴다.

명령	의미
<code>mixdog</code>	현재 프로젝트에서 대화형 세션 시작
<code>mixdog --workflow solo</code>	Solo 워크플로(기본). Lead가 위임 없이 직접 수행
<code>mixdog --readonly</code>	읽기 전용 도구만 사용
<code>mixdog --help</code>	전체 옵션 참조

표. 자주 쓰는 시작 명령. 전체 옵션은 `mixdog --help`. 출처: README.md Run 절.

1 README.md Get started / Desktop. Windows x64 설치 파일은 현재 unsigned.

2. 할 수 있는 일

저장소와 코드

텍스트와 심볼 검색으로 저장소를 찾고, 파일을 고치고, 테스트와 백그라운드 명령을 실행하고, 변경을 검토한다. Desktop은 Monaco 편집기, Git, 터미널, 파일 탐색기를 에이전트와 한자리에 둔다. GitHub 연동은 저장소, 이슈, 풀 리퀘스트와 리뷰, Actions, 릴리스, 알림을 다룬다. 소스 컨트롤 커밋 요약은 사용자가 직접 입력하며, 내장 AI 커밋 메시지 생성기는 없다.

긴 작업

저장한 대화를 재개하고, 로컬 의미·어휘 검색으로 이전 작업을 회상하며, 프로젝트 범위 선호를 세션 사이에 유지한다. Compaction이 긴 대화를 관리 가능한 크기로 유지한다. 명시적으로 요청한 장기 목표는 Goals로 완료 조건과 작업을 추적하고, 시간 제한과 자동 이어가기를 지원한다.

브라우저·데스크톱·문서·미디어

Browser Use는 로그인된 Chromium 페이지, 폼, 탭, 다운로드를 조작한다. Windows에서 Chrome 프로필 (쿠키·비밀번호) 가져오기는 관리자 승인과 네이티브 importer 빌드가 필요하다. 세션 쿠키는 OS 키체인으로 암호화된다. Computer Use는 Windows에서 접근성, 스크린샷, OCR, 키보드와 포인터로 네이티브 앱을 가드된 실행 정책 아래에서 조작한다. 문서는 Word, Excel, PowerPoint 작성·편집과 PDF, 렌더 미리보기와 품질 점검을 다룬다. 이미지·영상 Studio는 이미지 생성·편집, 짧은 영상, 로컬 갤러리를 제공한다. 클립 이어가기는 마지막 프레임을 참조하며 포즈는 이어지지만 원래 움직임이나 카메라 궤적은 이어지지 않는다.

Browser Use와 Computer Use는 선택 기능이다. 대화형 세션에서는 기본적으로 첫 실제 호출 전에 승인을 묻고, 그 승인은 해당 세션에 적용되며 재시작 시 다시 묻는다. Desktop, TUI, 페어 브라우저는 라이브 세션을 공유한다. 설치형 원격 웹 앱은 Desktop에 인증된 종단 간 암호화로 연결된다.

3. 효율 근거

Terminal-Bench 2.1²에서 같은 모델, 같은 89개 과제, 같은 공식 Harbor verifier로 하네스만 바꾸어 비교했다. 공식 리더보드 등재가 아니며, 실패와 타임아웃은 재시도하지 않는다. 비용은 구독 청구액이 아니라 동일 API 정가 환산이다.

항목	Mixdog	Codex CLI	해석
비용 (trial당)	\$0.476	\$0.782	39% 낮음
최종 context 중앙값	18.5k tokens	34.3k tokens	46% 작음
성공률 (k=5)	86.5% (385/445)	86.1% (383/445)	pass@5 96.6% vs 95.5%
시험 시간	415s	437s	속도 동등 수준

표. GPT-5.6 Sol xhigh, Mixdog vs Codex CLI. k=5, 각 445 trials. Mixdog trial당 \$0.476³. 출처: README.md Benchmarks.

항목	Mixdog	Claude Code	해석
비용 (run당)	\$104.29	\$129.21	19% 낮음
최종 context 중앙값	27.6k tokens	38.2k tokens	28% 작음
과제 통과	79/89	77/89	단일 패스(k=1)
시험 시간	610s	708s	1.16배 빠름

표. Claude Opus 5, Mixdog vs Claude Code. k=1, 89 trials. Mixdog run당 \$104.29⁴. 출처: README.md Benchmarks.

4. 제공자와 로컬 모델

Mixdog는 구독 OAuth와 API 키 경로를 지원한다. Anthropic API 키와 Claude 계정 OAuth, OpenAI API 키와 ChatGPT/Codex 계정 OAuth, Google Gemini API 키와 Antigravity OAuth, xAI API 키와 Grok 계정 OAuth, OpenRouter API 키, 실험적 Cursor 계정 OAuth, DeepSeek와 OpenCode Go, 그리고 내장 Local Provider를 쓸 수 있다. 이 목록은 임의 OpenAI 호환 엔드포인트 레지스트리가 아니다. 이전 Ollama와 LM Studio 경로는 은퇴했다.

2 README.md Benchmarks 절. 같은 89 tasks, 공식 Harbor verifier, fast mode off. Sol 비교는 양측 k=5(445 trials). Opus 비교는 k=1(89 trials). 핀된 소스 리비전 측정이며 공식 리더보드 등재가 아님.

3 README.md: GPT-5.6 Sol xhigh, Mixdog \$0.476 vs Codex CLI \$0.782 per trial. 동일 API 정가 환산. 최종 context 중앙값 18.5k vs 34.3k tokens.

4 README.md: Claude Opus 5, Mixdog \$104.29 vs Claude Code \$129.21 per run. 동일 API 정가 환산. 과제 통과 79/89 vs 77/89.

Local Provider는 별도 모델 서버 없이 Mixdog 안에서 모델을 받아 실행한다. 관리 런타임은 현재 Windows x64와 NVIDIA GPU, 선택한 모델을 담을 VRAM이 필요하다. 설치한 모델은 mixdog-local 제공자로 쓴다. 확장 → Plugin → Local Provider에서 설치·다운로드 재개·유휴 시 자동 언로드를 관리한다.

5. 사용 시 알아둘 점

Solo가 기본 워크플로이다. 병렬 위임은 Cowork(mixdog --workflow default)를 쓴다. 여러 Desktop 세션을 띄우는 일과 한 세션 안에서 에이전트에게 위임하는 일은 다르다.

mixdog exec는 비대화형 단일 모델 세션이다. 제공자와 모델을 명시해야 하며, 호스트의 행동 설정, 개인 기억, 이전 세션, 사용자 프로필, 스킬, MCP 서버, 플러그인을 불러오지 않는다. exec에서 웹 검색은 기본 꺼져 있으며, 꺼져 있어도 셸 네트워크(패키지 관리자, Git)는 막지 않는다. 오프라인 샌드박스가 아니다.

워크플로와 에이전트는 Markdown 정의 팩(WORKFLOW.md, AGENT.md)이다. 내장 팩은 Mixdog와 함께 제공되며, 사용자 팩은 Mixdog 데이터 디렉터리에 둔다.

6. 권장 첫 사용

1. Desktop 설치 파일 또는 npm으로 CLI를 설치한다.
2. 첫 실행에서 제공자를 인증하고 모델을 고른다.
3. 프로젝트 폴더에서 할 일을 지시한다. 기본 워크플로는 Solo다.
4. 브라우저나 Windows 앱 조작이 필요하다면 첫 호출 승인 후에 쓴다.
5. 오래 걸리는 목표는 Goal로 묶어 완료 조건을 명시한다.